

UNIT-1 Pointers and File Handling

Course- Fundamentals of Data Structures

Mr. Mandar K Mokashi

Department of Information Technology Engineering, VIIT, Pune-48



BRAC'T'S, Vishwakarma Institute of Information Technology, Pune-48

(An Autonomous Institute affiliated to Savitribai Phule Pune University)
(NBA and NAAC accredited, ISO 9001:2015 certified)

Objective/s of this session

1. To understand pointer fundamentals
2. To do pointer arithmetic

Learning Outcome/Course Outcome

1. Able to use pointers.
2. Able to perform pointer operations.

Content

Part –I

- Introduction to Pointers
- Dynamic memory allocation
- Pointer to pointer

Part-II

- Pointer to single and multidimensional arrays
- Array of pointers, String and structure manipulation using pointers
- Pointer to functions

Part-III

- Pointer to file structure
- basic operations on file
- File handling in C

Pe-requisite

Knowledge of

- Programming constructs: keywords, variables, constants, operators
- Types of instructions: declarative, I/O, arithmetic, control (decision making/ iterative)
- SDLC
- Array
- Structure
- Function: declaration, definition, call

UNIT I POINTERS AND FILE HANDLING

Introduction

- Pointers
 - Powerful, but difficult to master
 - Simulate call-by-reference
 - Close relationship with arrays and strings

What is pointer?

In a generic sense, a “pointer” is anything that tells us where something can be found.

- Addresses in the phone book
- URLs for webpages
- Road signs
- Page number in index

A pointer can contain the memory address of any variable type

- A primitive (int, char, float)
- An array
- A struct or union
- Dynamically allocated memory
- Another pointer
- A function

Why Pointers?

- They allow you to refer to large data structures in a compact way
 - Eg- array, database (array of structure elements)
- They facilitate sharing between different parts of programs
 - Call by reference in functions
- They make it possible to get new memory dynamically as your program is running
 - Dynamic memory allocation
- They make it easy to represent relationships among data items.
 - Complex data structures like linked list, stack, queue, tree, graph etc.

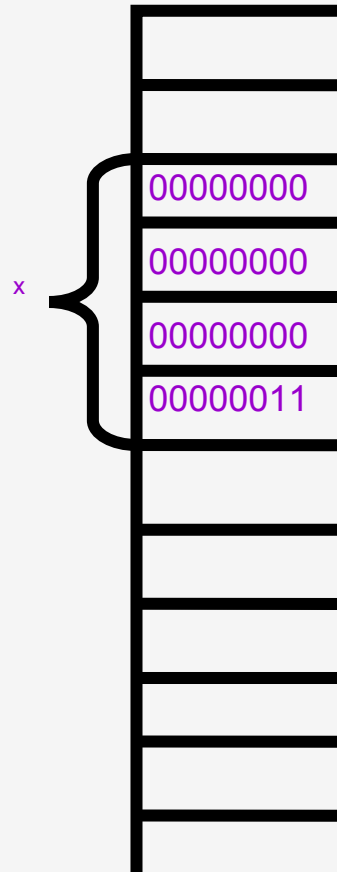
1. Pointer Fundamentals

PRIMARY OPERATORS

- When is & used?
 - & -- "address of operator" which gives or produces the memory address of a data variable
- When is * used?
 - -- (Value at address operator) "dereferencing operator" which provides the contents or value at address in the memory location specified by a pointer

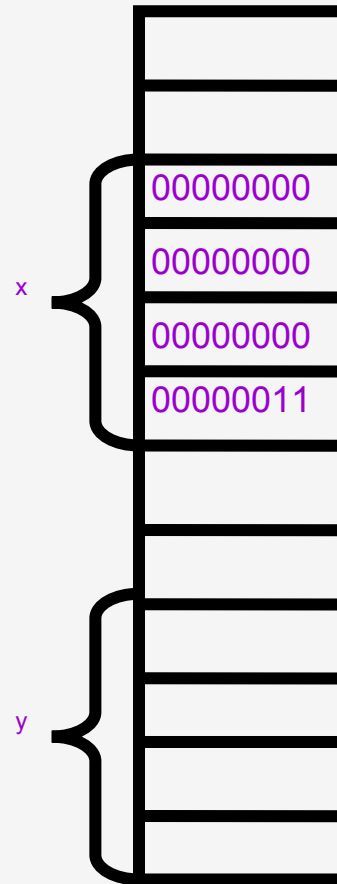
Pointers

- When a variable is defined, the compiler allocates a memory address for the variable.
 - `int x;` will allocate 4 bytes in the main memory, which will be used to store an integer value.
- When a value is assigned to a variable, the value is actually placed to the memory that was allocated.
 - `x=3;` will store integer 3 in the 4 bytes of memory.



Pointers

- When the value of a variable is used, the contents in the memory are used.
 - `y=x;` will read the contents in the 4 bytes of memory, and then assign it to variable `y`.
- `&x` can get the address of `x`. (referencing operator `&`)
- The address can be passed to a function:
 - `scanf("%d", &x);`
- The address can also be stored in a variable



Initializing Pointers

- To declare a pointer variable

```
type * pointer_name;
```

- For example:

- `int * p1;` `p1` is a variable that tends to point to an integer, (or `p1` is a `int` pointer)
- `char *p2;`
- `unsigned int * p3;`
- `p1 = &x;` `/* Store the address in p1 */`
- `scanf("%d", p1);` `/* i.e. scanf("%d",&x); */`

Initializing Pointers

- Like other variables, always initialize pointers before using them!!!
- For example:

```
int main(){
```

```
    int x;
```

```
    int *p;
```

```
    scanf("%d",p); /*
```



```
*/
```

```
    p = &x;
```

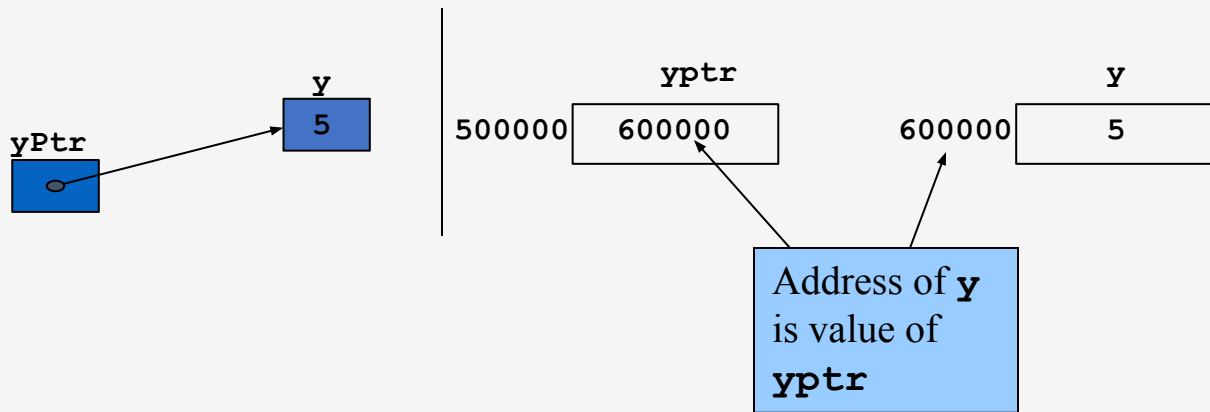
```
    scanf("%d",p); /* Correct */
```

```
}
```

Pointer Operators

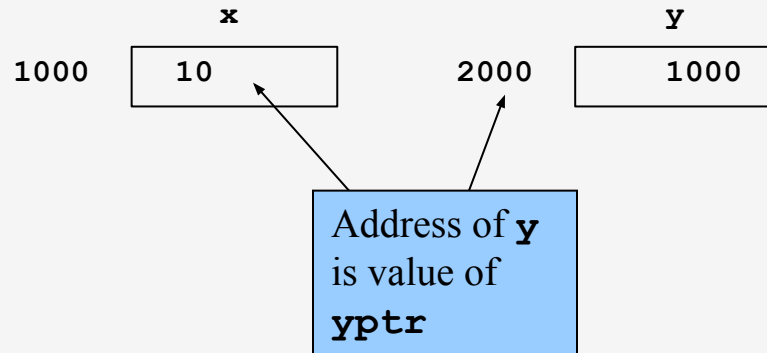
- & (address of operator)
 - Returns address of operand

```
int y = 5;
int *yPtr;
yPtr = &y;    // yPtr gets address of y
yPtr "points to" y
```



Pointer Operators

```
int x=10;  
Int *y=&x;  
Printf("%d",x);  
Printf("%d",&x);  
Printf("%d",y);  
Printf("%d",*y);  
Printf("%d",&y);
```



Advice and Precaution

- Pros
 - Efficiency
 - Convenience
- Cons
 - Error-prone
 - Difficult to debug

Dynamic Memory Allocation in C

In *static memory management*, the memory requirement is fixed. It could not be increased or decreased during execution of the program. This type of allocation has a drawback. We cannot predict how much memory will be required. For example, if we declare an array, like this: the process of allocating memory at runtime is known as **dynamic memory allocation**. Library functions known as **memory management functions** are used for assigning (allocating) and freeing memory, during execution of a program.

These functions are defined in the **stdlib.h** header file.

These functions allocate memory from a memory area known as **heap** and when the memory is not in use, it is freed to be used for some other purpose.

We access the dynamically allocated memory using [pointers](#)

Dynamic Memory Allocation in C

```
int marks[50];
```

We might only store data for **25 students**. As a result, memory will be wasted. Or we may need to store data for **60 students**, in that case we will have less memory.

The solution to these problems is dynamic memory allocation.

Difference between Static and Dynamic Memory Allocation

Static Memory Allocation	Dynamic Memory Allocation
Allocation is done before execution of the program(at compile time).	Allocation is done during execution of the program(at runtime).
Here, variables are allocated memory permanently.	Here, memory is allocated only when the program is active.
It uses stack for managing memory allocation.	It uses heap for managing memory allocation.
We cannot reuse the memory.	We can reuse the memory and free it after usage.
Execution is faster than dynamic memory allocation.	Execution is slower than static memory allocation.
It is less efficient.	It is more efficient.
It is simple to use.	It can be complicated to use.

Dynamic Memory Allocation techniques

Function	Description
malloc()	allocates requested size of bytes and returns a void pointer pointing to the first byte of the allocated space
calloc()	allocates space for an array of elements, initialize them to zero and then returns a void pointer to the memory
free	releases previously allocated memory
realloc	modify the size of previously allocated space

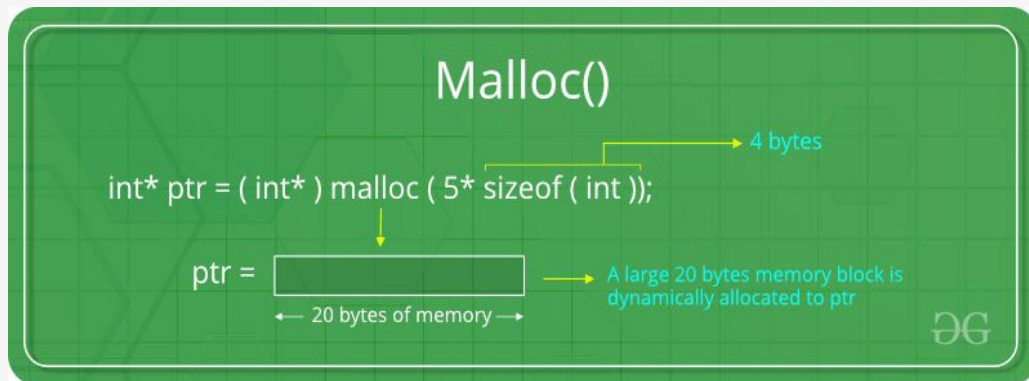
Dynamic Memory Allocation

1. malloc() Function

malloc() function is used for allocating block of memory at runtime. This function reserves a block of memory of the given size and returns a **pointer** of type void. This means that we can assign it to any type of pointer using typecasting. If it fails to allocate enough space as specified, it returns a NULL pointer.

Syntax:

```
void* malloc(size_t size)
data_type *ptr;
ptr = (data_type *)malloc(specified size);
```



Dynamic Memory Allocation

```
void main(){
    int *x;
    float *y;
    x = (int*)malloc(50);    //50 contiguous bytes of memory are allocated and the
                           //address of the first byte is stored in x.

    y = (float*)malloc(5 * sizeof(float));    // This allocates memory to store 5 decimal
                                             // numbers.

    if(x == NULL)                // we should always check the value returned.
        printf("Memory unavailable\n");
    if(y == NULL)
        printf("Memory unavailable\n");
}
```

Dynamic Memory Allocation

```
#include<stdio.h>
#include <stdlib.h>
int main(){
    int *ptr;
    ptr = malloc(5 * sizeof(int));           /* a block of 5 integers */
    if (ptr != NULL)
    {
        *(ptr + 1) = 10;                   /* assign 10 to second integer */

        printf("Value of the 2nd integer is %d",*(ptr + 1));
    }
    return 0;
}
```

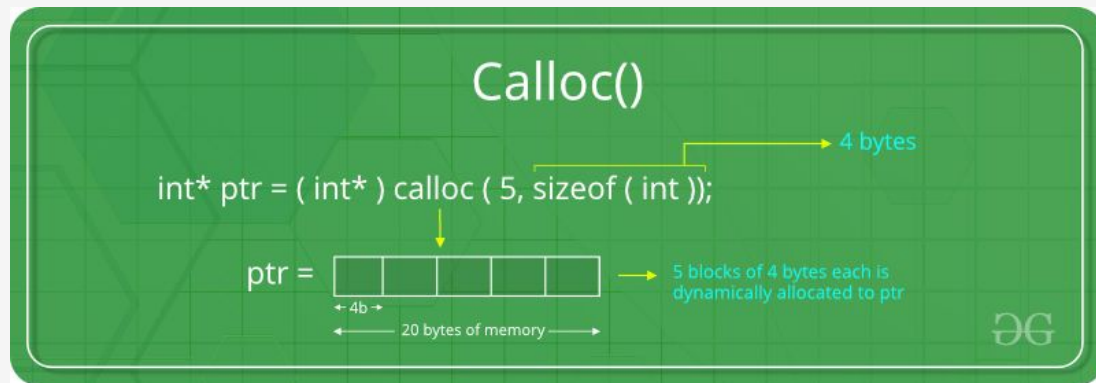
Dynamic Memory Allocation

2. calloc() Function

calloc function is normally used for allocating memory to derived data types such as arrays and structures.

Ex.

```
struct employee
{
    char *name;
    int salary;
};
typedef struct employee emp;
emp *e1;
e1 = (emp*)calloc(30, sizeof(emp));
```



Dynamic Memory Allocation

```
int main()
{
    // This pointer will hold the base address of the block created
    int* ptr;
    int n, i;
    printf("Enter number of elements:");           // Get the number of elements for the array
    scanf("%d",&n);
    printf("Entered number of elements: %d\n", n);
    ptr = (int*)malloc(n * sizeof(int));           // Dynamically allocate memory using malloc()
    // Check if the memory has been successfully allocated by malloc or not
    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {
        // Memory has been successfully allocated
        printf("Memory successfully allocated using malloc.\n");
        // Get the elements of the array
        for (i = 0; i < n; ++i) {
            ptr[i] = i + 1;
        }

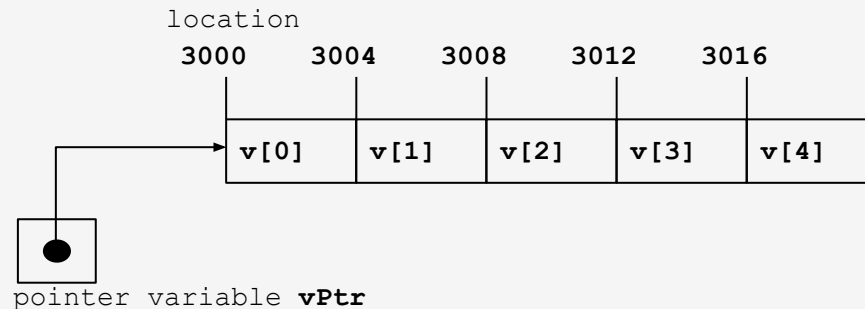
        // Print the elements of the array
        printf("The elements of the array are: ");
        for (i = 0; i < n; ++i) {
            printf("%d, ", ptr[i]);
        }
        return 0;
    }
}
```

Pointer Expressions and Pointer Arithmetic

- Arithmetic operations can be performed on pointers
 - Increment/decrement pointer (`++` or `--`)
 - Add an integer to a pointer(`+` or `+=` , `-` or `-=`)
 - Pointers may be subtracted from each other

Pointer Expressions and Pointer Arithmetic

- 5 element `int` array on machine with 4 byte `ints`
 - `vPtr` points to first element `v[ 0 ]`
 - at location 3000 (`vPtr = 3000`)
 - `vPtr += 2 ;` sets `vPtr` to 3008
 - `vPtr` points to `v[ 2 ]` (incremented by 2), but the machine has 4 byte `ints`, so it points to address 3008

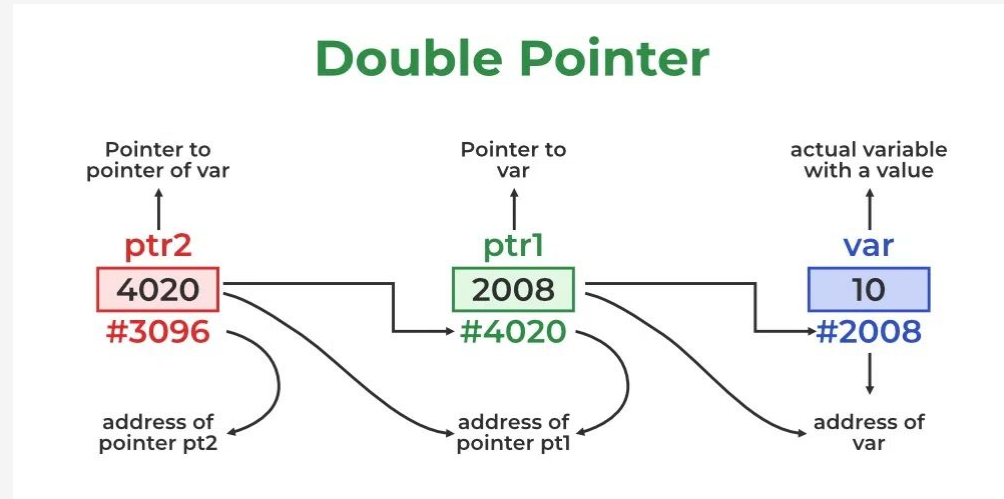


3. Pointer to pointer

C - Pointer to Pointer

- When we define a pointer to a pointer, the first pointer contains the address of the second pointer, which points to the location that contains the actual value as shown below.

- `int **var;`



C - Pointer to Pointer

```
int main () {  
  
    int var;  
    int *ptr;  
    int **pptr;  
  
    var = 3000;  
  
    /* take the address of var */  
    ptr = &var;  
  
    /* take the address of ptr using address of operator & */  
    pptr = &ptr;  
  
    /* take the value using pptr */  
    printf("Value of var = %d\n", var );  
    printf("Value available at *ptr = %d\n", *ptr );  
    printf("Value available at **pptr = %d\n", **pptr);  
  
    return 0;  
}
```

C - Pointer to Pointer

```
int main () {  
  
    int var;  
    int *ptr;  
    int **pptr;  
  
    var = 3000;  
  
    /* take the address of var */  
    ptr = &var;  
  
    /* take the address of ptr using address of operator & */  
    pptr = &ptr;  
  
    /* take the value using pptr */  
    printf("Value of var = %d\n", var );  
    printf("Value available at *ptr = %d\n", *ptr );  
    printf("Value available at **pptr = %d\n", **pptr);  
  
    return 0;  
}
```

Output=

Value of var = 3000

Value available at *ptr = 3000

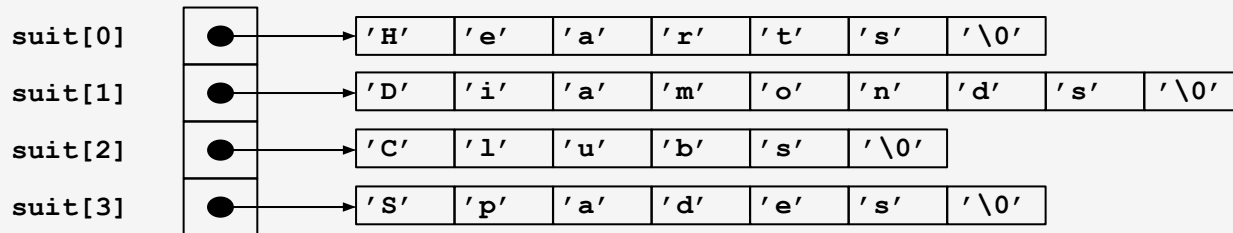
Value available at **pptr = 3000

Arrays of Pointers

- An array of pointers is an array that consists of variables of pointer type, which means, the variable is a pointer addressing to some other element.
- For example: an array of strings

```
char *suit[ 4 ] = { "Hearts", "Diamonds", "Clubs", "Spades" };
```

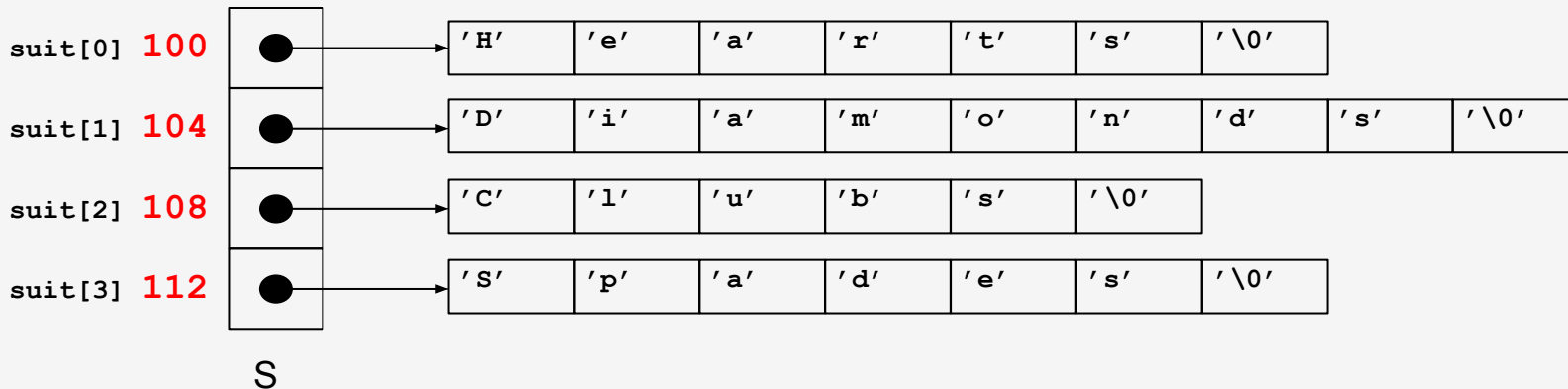
- Strings are pointers to the first character
- **char *** – each element of **suit** is a pointer to a **char**
- The strings are not actually stored in the array **suit**, only pointers to the strings are stored



- **suit** array has a fixed size, but strings can be of any size

Arrays of Pointers

- ❑ Int main()
- ❑ Char *S[4]={“Heart”,“Diamond”,“Club”,“Spades”};
- ❑ Printf(“String=%s”,*s+1); ----- earts
- ❑ Printf(“String=%s”,*(s+2)); -----clubs
- ❑ Printf(“String=%s”,(*s+1)+2);---amond



Pointer to Functions

TYPES OF FUNCTION CALLS

- **Call by Value:**

When a function is called by an argument/parameter which is not a pointer, the copy of ***the argument*** is passed to the function.

Therefore a possible change on the copy does not change the original value of the argument.

Example

Program to calculate and print the area and the perimeter of a circle Using **Call by value** approach

```
#include<stdio.h> /*The function calls are Call by Value*/
```

```
#define pi 3.14
```

```
float area(float);
```

```
float perimeter(float);
```

```
int main( )
```

```
{  
    float r, a, p;  
    printf("Enter the radius\n");
```

```
    scanf("%f",&r);
```

```
    a = area(r);
```

```
    p = perimeter(r);
```

```
    printf("The area = %f, \n The Perimeter = %f", a, p);
```

```
    return 0;
```

```
}
```

```
float area(float x)
```

```
{
```

```
    return pi*x*x;
```

```
}
```

```
float perimeter(float y)
```

```
{
```

```
    return 2.0*pi*y;
```

```
}
```

TYPES OF FUNCTION CALLS

- **Call by Reference:**

- If instead of passing the values of the variables to the called function, we pass their addresses, so that the called function can change the values stored in the calling routine. This is known as "call by reference" since we are referencing the variables.

Example

Program to calculate and print the area and the perimeter of a circle using *Call by reference* approach

```
#include<stdio.h> /*The function calls is Call by Reference*/
```

```
#define pi 3.14
```

```
void area_perimeter(float, float *, float *);
```

```
int main( )
```

```
{
```

```
float r, a, p;
```

```
printf("Enter the radius\n");
```

```
scanf("%f",&r);
```

```
area_perimeter(r,&a,&p);
```

```
printf("The area = %f, \n The Perimeter = %f", a, p);
```

```
return 0;
```

```
}
```

```
void area_perimeter(float x, float *aptr, float *pptr);
```

```
{
```

```
*aptr = pi*x*x;
```

```
*pptr = 2.0*pi*x;
```

```
}
```

Swapping two numbers

```
#include <stdio.h>

void swap ( int a, int b ) ;

int main ( )
{
    int a = 5, b = 6;
    printf("a=%d b=%d\n",a,b) ;
    swap (a, b) ;
    printf("a=%d b=%d\n",a,b) ;
    return 0 ;
}
```

```
void swap( int a, int b )
{
    int temp;
    temp= a; a= b; b = temp ;
    printf ("a=%d b=%d\n", a, b);
}
```

Results:

a=5 b=6

a=6 b=5

a=5 b=6 ****

Pointers with Functions (example)

```
#include <stdio.h>
void swap ( int *a, int *b ) ;
int main ( )
{
    int a = 5, b = 6;
    printf("a=%d b=%d\n",a,b) ;
    swap (&a, &b) ;
    printf("a=%d b=%d\n",a,b) ;
    return 0 ;
}
```

```
void swap( int *a, int *b )
{
    int temp;
    temp= *a; *a= *b; *b = temp ;
    printf ("a=%d b=%d\n", *a, *b);
}
```

Results:

a=5 b=6

a=6 b=5

a=6 b=5 *//permanent change*

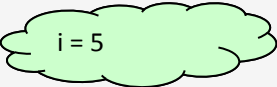
Example: Pass by Reference

- Modify behaviour in argument passing

```
void f(int j)
{
    j = 5;
}
void g()
{
    int i = 3;
    f(i);
} . . .
```



```
void f(int *ptr)
{
    *ptr = 5;
}
void g()
{
    int i = 3;
    f(&i);
} . . .
```



Pointers as Function Parameters

- Sometimes, you want a function to assign a value to a variable. E.g. you want a function that computes the minimum AND maximum numbers in 2 integers.
- Method 1, use two global variables.
 - In the function, assign the minimum and maximum numbers to the two global variables.
 - When the function returns, the calling function can read the minimum and maximum numbers from the two global variables.
- This is bad because the function is not reusable.

Pointers as Function Parameters

- Instead, we use the following function

```
void min_max(int a, int b,  
             int *min, int *max){  
    if(a>b){  
        *max=a;  
        *min=b;  
    }  
    else{  
        *max=b;  
        *min=a;  
    }  
}
```

```
int main()  
{  
    int x,y;  
    int small,big;  
    printf("Two integers: ");  
    scanf("%d %d", &x, &y);  
    min_max(x,y,&small,&big);  
    printf("%d <= %d", small, big);  
    return 0;  
}
```

Calling Functions by Reference

- Call by reference with pointer arguments
 - Pass address of argument using **&** operator
 - Allows you to change at actual location in memory
 - Arrays are not passed with **&** because the array name is already a pointer

5. Function pointers

What are function Pointers?

- C does not require that pointers only point to data, it is possible to have pointers to functions
- Functions occupy memory locations therefore every function has an address just like each variable

Pointers to Functions

- Pointer to function/ function pointers
 - Contains address of function
 - Similar to how array name is address of first element
 - Function name is starting address of code that defines function
- Function pointers can be
 - Passed to functions
 - Stored in arrays
 - Assigned to other function pointers

Calling a function using a Function Pointer

- There are two alternatives
 - 1) Use the name of the function pointer
 - 2) Can explicitly dereference it

```
int (*funcPointer) (int, char, int);
```

```
// calling a function using function pointer
```

```
int answer= funcPointer (7, 'A' , 2 );
```

```
int answer=(* funcPointer) (7, 'A' , 2 );
```

6. Structures and pointers

Pointer to Structure

```
void main()
{
    struct book
    {
        char name[25];
        char author[25];
        int callno;
    };
    struct book b1={"Let us c","Yaswant Kanetkar",101};
    struct book *ptr;
    ptr=&b1;
    printf("%s %s %d",b1.name,b1.author,b1.callno);
    printf("\n %s %s %d",ptr->name,ptr->author,ptr->callno);
}
```

Passing address of a structure variable

```
struct book
{
    char name[25];
    char author[25];
    int callno;
};

void display(struct book*);

void main()
{
    struct book b1={"Let us c","Yaswant Kanetkar",101};
    display(&b1);
}

void display(struct book *b)
{
    printf("\n %s %s %d",b->name,b->author,b->callno);
}
```

Instructions/Guidelines/References

Practice the problem statements given in the exercises of

1. Pointers in C, Yashwant Kanitkar
2. Let us C, Yashwant Kanitkar

Part-III

- Pointer to file structure
- basic operations on file
- File handling in C

File handling in C

File handling in C is the process in which we create, open, read, write, and close operations on a file. C language provides different functions such as `fopen()`, `fwrite()`, `fread()`, `fseek()`, `fprintf()`, etc. to perform input, output, and many different C file operations

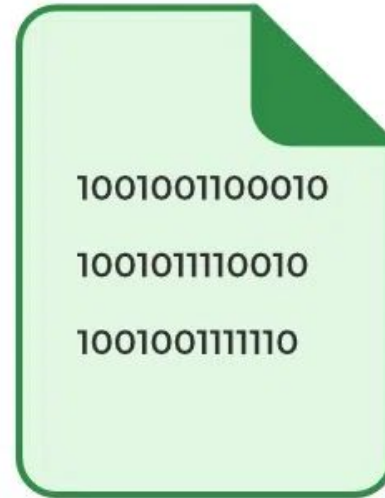
Why do we need File Handling in C?

- **Reusability:** The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- **Portability:** Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- **Efficient:** A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- **Storage Capacity:** Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

Types of Files in C



file.txt



file.bin

Types of Files in C

- **1. Text Files**

- A text file contains data in the **form of ASCII characters** and is generally used to store a stream of characters.
- Each line in a text file ends with a new line character ('\n').
- It can be read or written by any text editor.
- They are generally stored with **.txt** file extension.
- Text files can also be used to store the source code.

- **2. Binary Files**

- A binary file contains data in **binary form (i.e. 0's and 1's)** instead of ASCII characters. It is stored in the main memory.
- The binary files can be created only from within a program and their contents can only be read by a program.
- More secure as they are not easily readable.
- They are generally stored with **.bin** file extension.

C File Operations

- Different possible operations that we can perform on a file in C :
- Creating a new file – **fopen()** with attributes as “a” or “a+” or “w” or “w+”
- Opening an existing file – **fopen()**
- Reading from file – **fscanf()** or **fgets()**
- Writing to a file – **fprintf()** or **fputs()**
- Moving to a specific location in a file – **fseek()**, **rewind()**
- Closing a file – **fclose()**

File operation	Declaration & Description
fopen() - To open a file	Declaration: FILE *fopen (const char *filename, const char *mode) fopen() function is used to open a file to perform operations such as reading, writing etc. In a C program, we declare a file pointer and use fopen() as below. fopen() function creates a new file if the mentioned file name does not exist. <pre>FILE *fp; fp=fopen ("filename", "'mode");</pre> Where, fp - file pointer to the data type "FILE". filename - the actual file name with full path of the file. mode - refers to the operation that will be performed on the file. Example: r, w, a, r+, w+ and a+. Please refer below the description for these mode of operations.
fclose() - To close a file	Declaration: int fclose(FILE *fp); fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below. <pre>fclose (fp);</pre>
fgets() - To read a file	Declaration: char *fgets(char *string, int n, FILE *fp) fgets function is used to read a file line by line. In a C program, we use fgets function as below. <pre>fgets (buffer, size, fp);</pre> where, buffer - buffer to put the data in. size - size of the buffer fp - file pointer
fprintf() - To write into a file	Declaration: int fprintf(FILE *fp, const char *format, ...); fprintf() function writes string into a file pointed by fp. In a C program, we write string into a file as below. fprintf (fp, "some data"); or fprintf (fp, "text %d", variable_name);

File Pointer in C

A file pointer is a reference to a particular position in the opened file. It is used in file handling to perform all file operations such as read, write, close, etc.

Syntax of File Pointer

```
FILE* pointer_name;
```

Open a File in C

For opening a file in C, the [`fopen\(\)`](#) function is used with the filename or file path along with the required access modes.

Syntax of `fopen()`

```
FILE* fopen(const char *file_name, const char *access_mode);
```

Parameters

file_name: name of the file when present in the same directory as the source file. Otherwise, full path.

access_mode: Specifies for what operation the file is being opened.

Return Value

If the file is opened successfully, returns a file pointer to it.

If the file is not opened, then returns NULL.

File opening modes in C

File opening modes or access modes specify the allowed operations on the file to be opened. They are passed as an argument to the `fopen()` function. Some of the commonly used file access modes are listed below:

File Pointer in C

```
// C Program to illustrate file opening
#include <stdio.h>
#include <stdlib.h>
int main()
{
    // file pointer variable to store the value returned by fopen
    FILE* fptr;
    // opening the file in read mode
    fptr = fopen("filename.txt", "r");
    // checking if the file is opened successfully
    if (fptr == NULL) {
        printf("The file is not opened. The program will now exit.");
        exit(0);
    }
    return 0;
}
```

Reading Data from an existing file

We can read content of a file in c using the fscanf(), fgets() and fgetc() functions.

- **fscanf()**
- The fscanf() function is used to read character set i.e strings from the file. It returns the EOF, when all the content of the file are read by it.
- **Syntax**
- `int fscanf(FILE *stream, const char *charPointer[])`
- **Parameters**
- `FILE *stream`: the pointer to the opened file. `const char *charPointer[]`: string of character.

Reading Data from an existing file

```
#include <stdio.h>

int main(){
    FILE * file;
    char str[500];
    if (file = fopen("hello.txt", "r")){
        while(fscanf(file,"%s", str)!=EOF){
            printf("%s", str);
        }
    }
    else
        printf("Error!");
    fclose(file);
    return 0;
}
```

Output

LearnprogrammingattutorialsPoint

Reading Data from an existing file

fgets()

The `fgetc()` function in C is used to read string from the stream.

Syntax

```
char* fgets(char *string, int length, FILE *stream)
```

Parameter

`char *string`: It is a string which will store the data from the string. `int length`: It is an int which gives the length of string to be considered. `FILE *stream`: It is the pointer to the opened file.

```
#include <stdio.h>
```

```
int main(){
```

```
    FILE * file;
```

```
    char str[500];
```

```
    if (file = fopen("hello.txt", "r")){
```

```
        printf("%s", fgets(str, 50, file));
```

```
    }
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

Reading Data from an existing file

fgetc()

Return a single character from the file. It gets a character from the file and returns EOF at the end of file.

Syntax

```
char* fgetc(FILE *stream)
```

Parameter - FILE *stream: It is the pointer to the opened file.

Example

```
#include <stdio.h>

int main(){
    FILE * file;
    char str;
    if (file = fopen("hello.txt", "r")){
        while((str=fgetc(file))!=EOF)
            printf("%c",str);
    }
    fclose(file);
    return 0;
}
```

Writing Data to a file in C

We can write data to a file in C using the `fprintf()`, `fputs()`, `fputc()` functions. All are used to write contents to a file.

fprintf()

The `fprintf()` function is used to write data to a file. It writes a set of characters in a file.

Syntax

```
int fprintf(FILE *stream, char *string[])
```

Parameters

FILE for `*stream`: It is the pointer to the opened file.

char `*string[]`: It is the character array that we want to write in the file.

Writing Data to a file in C

```
#include <stdio.h>

int main(){
    FILE * file;
    if (file = fopen("hello.txt", "w")){
        if(fprintf(file, "tutorials Point") >= 0)
            printf("Write operation successful");
    }
    fclose(file);
    return 0;
}
```

Output

Write operation successful

Writing Data to a file in C

- `fputf()`
- The `fputf()` function in C can be used to write to a file. It is used to write a line (character line) to the file.
- Syntax
- `int fputs(const char *string, FILE *stream)`Parameters
- `Constant char *string[]`: It is the character array that we want to write in the file. `FILE for *stream`: It is the pointer to the opened file.

Writing Data to a file in C

```
#include <stdio.h>

int main(){
    FILE * file;
    if (file = fopen("hello.txt", "w")){
        if(fputs("tutorials Point", file) >= 0)
            printf("String written to the file successfully...");
    }
    fclose(file);
    return 0;
}
```

Output

String written to the file successfully...

Writing Data to a file in C

- fputc()
- The fputc() function is used to write a single character to the file.
- Syntax
- `int fputc(char character , FILE *stream)`Parameters
- `char character` : It is the character that we want to write in the file.
`FILE for *stream`: It is the pointer to the opened file.

Output

String written to the file successfully...

Writing Data to a file in C

```
#include <stdio.h>

int main(){
    FILE * file;
    if (file = fopen("hello.txt", "w")){
        fputc('T', file);
    }
    fclose(file);
    return 0;
}
```

Output

'T' is written to the file.